03 · Experiment Tracking & Model Versioning

> **Goal:** make every model reproducible and comparable: *which code, data, parameters and environment produced which metric and artifact?*

■ Decision Tree: which component, and when?

■ How this module intersects the others

> Full course map: assets/tree_00_master_map.svg

Contents 1. What must be tracked 2. MLflow tracking at scale 3. Weights & Biases 4. DVC: data & pipeline versioning 5. Reproducibility checklist 6. Edge cases & production failures 7. Senior interview questions & answers 8. Tips, Tricks & Field Notes 9. Worked Scenario: The best run cannot be reproduced 10. More Interview Questions 11. Where This Module Connects 12. Interview Playbook: Tips & Tricks 13. Scenario-Based Evaluation 14. Rapid-Fire Round - 3.15 Statistical Rigor in Model Comparison - 3.16 Metadata, Lineage and Reproducibility at Scale

---

1. What must be tracked

Dimension	Examples	Where stored
Code	git SHA, dirty flag, diff	MLflow tag `mlflow.source.git.commit`
Data	dataset hash/DVC rev, row count, time window	DVC `.lock`, MLflow tag/dataset input
Params	hyperparameters, feature list, seed	MLflow params
Environment	Docker image digest, `pip freeze`, CUDA/driver	Artifact / tag
Metrics	train/val/test, per-slice, over steps	MLflow metrics
Artifacts	model, plots, confusion matrices, SHAP	Artifact store (S3/GCS/MinIO)
Signature	input/output schema	MLflow model signature



2. MLflow tracking at scale

Server for teams (not `mlruns/` on a laptop)

`docker-compose.mlflow.yml`

> Versions pinned in this chapter's snippets (MLflow v2.16.2, unpinned `minio/minio`) were current when written and will age; check current documentation and pin deliberately.

```
services:
  postgres:
    image: postgres:16
    environment: {POSTGRES_USER: mlflow, POSTGRES_PASSWORD: mlflow, POSTGRES_DB: mlflow}
    volumes: [pgdata:/var/lib/postgresql/data]
  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment: {MINIO_ROOT_USER: minio, MINIO_ROOT_PASSWORD: minio12345}
    ports: ["9000:9000", "9001:9001"]
    volumes: [minio:/data]
  mlflow:
    image: ghcr.io/mlflow/mlflow:v2.16.2
    depends_on: [postgres, minio]
    environment:
      MLFLOW_S3_ENDPOINT_URL: http://minio:9000
      AWS_ACCESS_KEY_ID: minio
      AWS_SECRET_ACCESS_KEY: minio12345
    command: >
      sh -c "pip install -q psycopg2-binary boto3 &&
             mlflow server --host 0.0.0.0 --port 5000
             --backend-store-uri postgresql://mlflow:mlflow@postgres/mlflow
             --artifacts-destination s3://mlflow --serve-artifacts"
    ports: ["5000:5000"]
volumes: {pgdata: {}, minio: {}}
```

(Create the `mlflow` bucket in MinIO once; for production use real secrets, TLS and authentication.)

A tracking decorator that enforces lineage

```
import functools, hashlib, os, platform, subprocess, sys
import mlflow

def _git(*args) -> str:
    try:
        return subprocess.check_output(["git", *args], text=True).strip()
    except Exception:
        return "unknown"

def tracked(experiment: str, data_path: str | None = None, tags: dict | None = None):
    """Wrap a training function so every run records code/data/env lineage.
    The wrapped function returns (model, metrics: dict, params: dict)."""
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*a, **kw):
            mlflow.set_experiment(experiment)
            with mlflow.start_run(run_name=fn.__name__) as run:
                mlflow.set_tags({
                    "git.sha": _git("rev-parse", "HEAD"),
                    "git.dirty": str(bool(_git("status", "--porcelain"))),
                    "python": platform.python_version(),
                    "image": os.getenv("IMAGE_DIGEST", "local"),
                    **(tags or {}),
                })
                if data_path:
                    hasher = hashlib.sha256()
                    with open(data_path, "rb") as fh:                # chunked: do not load bi
g files into RAM
                        for chunk in iter(lambda: fh.read(1 << 20), b""):
                            hasher.update(chunk)
                    h = hasher.hexdigest()
                    mlflow.set_tag("data.sha256", h)
                model, metrics, params = fn(*a, **kw)
                mlflow.log_params(params)
                mlflow.log_metrics(metrics)
```

```
            mlflow.log_text(subprocess.check_output(
                [sys.executable, "-m", "pip", "freeze"], text=True), &q
uot;env/requirements.txt")
            return run.info.run_id, model
        return wrapper
    return deco
```

Usage with signature + input example (required for safe serving):

```
import mlflow.sklearn
from mlflow.models import infer_signature
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score

@tracked("fraud-detection", data_path="data/train.csv", tags={"owner": &quo
t;risk-ml"})
def train(X_tr, y_tr, X_va, y_va, lr=0.05, n=400):
    m = GradientBoostingClassifier(learning_rate=lr, n_estimators=n, random_state=7).fit(X_tr, y_tr)
    auc = roc_auc_score(y_va, m.predict_proba(X_va)[:, 1])
    sig = infer_signature(X_va, m.predict_proba(X_va))
    mlflow.sklearn.log_model(m, "model", signature=sig, input_example=X_va.head(3))   # newer M
Lflow (3.x) prefers a `name=` argument: check current docs
    return m, {"val_auc": auc}, {"lr": lr, "n_estimators": n}
```

Querying and comparing runs

```
import mlflow
runs = mlflow.search_runs(
    experiment_names=["fraud-detection"],
    filter_string="metrics.val_auc > 0.9 and tags.`git.dirty` = 'False'",
    order_by=["metrics.val_auc DESC"], max_results=10)
best = runs.iloc[0]
print(best.run_id, best["metrics.val_auc"])
```

Metric logging patterns

- **Step-wise** curves: `mlflow.log_metric("loss", v, step=epoch)`.
- **Slice metrics**: val_auc.region=eu, naming convention allows dashboards by slice.
- **Large artifacts**: log once; reference by URI. Do not log 1 GB checkpoints every epoch.
- **System metrics**: `mlflow.enable_system_metrics_logging()` for GPU/CPU/mem.
- **Autolog with care**: `mlflow.autolog()` is convenient but logs many params; pin what you consider contract.

3. Weights & Biases

W&B shines for rich visualisation, sweeps and team collaboration; MLflow shines for self-hosting and registry integration.

Capability	MLflow	W&B
Hosting	Self-host OSS (or managed)	SaaS / dedicated / self-managed
Experiment UI	Good	Excellent (panels, media, comparisons)
Hyperparameter sweeps	Via Optuna/Hyperopt integration	Native sweeps (Bayesian, early-stop)
Registry	Built-in	W&B Registry/Artifacts
Cost	Infra + ops	Seat/usage based
Data residency	Full control	Plan dependent

```
import wandb

sweep_cfg = {
```

```
    "method": "bayes",
    "metric": {"name": "val_auc", "goal": "maximize"},
    "parameters": {
        "lr": {"distribution": "log_uniform_values", "min": 1e-4,
 "max": 1e-1},
        "max_depth": {"values": [4, 6, 8, 10]},
    },
    "early_terminate": {"type": "hyperband", "min_iter": 3},
}

def run():
    with wandb.init(project="fraud") as r:
        cfg = r.config
        auc = train_and_eval(lr=cfg.lr, max_depth=cfg.max_depth)  # your function
        wandb.log({"val_auc": auc})
        art = wandb.Artifact("fraud-model", type="model")
        art.add_file("artifacts/model.joblib")
        r.log_artifact(art)

sweep_id = wandb.sweep(sweep_cfg, project="fraud")
wandb.agent(sweep_id, function=run, count=30)
```

4. DVC: data & pipeline versioning

Git cannot store large data. DVC stores small **pointer files** (.dvc, dvc.lock) in Git and the data in remote storage, content-addressed by hash.

```
Git repo                    DVC remote (S3/GCS/Azure/SSH)
■■ data/raw.csv.dvc  ■■hash■■■  ab/cdef...  (the actual bytes)
■■ dvc.yaml  (pipeline)
■■ dvc.lock  (hashes of deps/outs per stage)
■■ params.yaml
```

```
dvc init
dvc remote add -d storage s3://my-bucket/dvcstore
dvc add data/raw/train.csv          # creates data/raw/train.csv.dvc + .gitignore entry
git add data/raw/train.csv.dvc data/raw/.gitignore .dvc/config
git commit -m "data: add train v1"
dvc push                            # upload bytes

# time travel to reproduce an old model
git checkout <old-sha> && dvc checkout   # restores matching data

# compare experiments
dvc params diff main
dvc metrics diff main
dvc exp run -S train.max_depth=12        # lightweight experiment, no commit needed
dvc exp show
```

Linking DVC to MLflow so a run points to exact data:

```
import subprocess
import mlflow
rev = subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).stri
p()
lock = open("dvc.lock").read()
mlflow.set_tag("dvc.git_rev", rev)
mlflow.log_text(lock, "lineage/dvc.lock")
```

Choosing a versioning approach

Need	DVC	lakeFS	Delta/Iceberg time travel
Small–mid files in Git workflow	■	■	■
Petabyte lake, branching	■	■	■
Table-level snapshot, SQL	■	■	■
Pipeline DAG with caching	■	✗	✗

5. Reproducibility checklist

1. **Pin dependencies** with a lock file (`uv.lock`, `pip-compile`, `conda-lock`) and build from a **container image digest** — not `latest`.
2. **Seed everything**: Python `random`, NumPy, framework (`torch.manual_seed`, `torch.use_deterministic_algorithms(True)` when feasible), data loaders' worker seeds.
3. **Version data** (DVC rev / table snapshot id) and record row counts and hashes.
4. **One entrypoint** `python -m project.train --config configs/x.yaml`; no manual notebook steps.
5. **Config as code**: typed, committed; log the *resolved* config.
6. **Record hardware**: GPU model, driver, CUDA, cuDNN; nondeterministic kernels can alter results.
7. **Re-run test**: CI nightly reruns a small config and asserts metric within tolerance.

```
import os, random, numpy as np, torch

def seed_everything(seed: int = 42, deterministic: bool = True):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    if deterministic:
        os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
        torch.use_deterministic_algorithms(True, warn_only=True)
        torch.backends.cudnn.benchmark = False
```

6. Edge cases & production failures

Failure	Root cause	Impact	Fix
"Best" run unreproducible	Dirty working tree / uncommitted notebook change	Cannot rebuild champion	Block registration when `git.dirty=True`; CI-only promotion
Tracking server DB bloat	Per-step metrics at high frequency, millions of rows	Slow UI/queries	Log every N steps, aggregate; partition/retention; move to Postgres
Artifact store permission drift	Training job role lacks `s3:PutObject` after IAM change	Runs "succeed" but model missing	Verify artifact existence at end of run; fail loudly
Metric cherry-picking	Selecting best test score among 200 runs	Optimistic estimate (leakage through selection)	Select on validation; final test touched once; report multiple-comparison caveat
Pickle incompatibility	Train sklearn 1.3, serve 1.5	Load error or silent change	Log env + conda.yaml; serve using MLflow `pyfunc` in same image; use ONNX for portability
Data changed under same path	Overwritten file in S3	Old runs referencing path no longer reproducible	Immutable content-addressed storage; version in path
Non-determinism	Multi-threaded float reductions, GPU atomics	ΔAUC ~0.002 between runs	Track variance across seeds; compare with CIs not point estimates

7. Senior interview questions & answers

Q1. How would you make sure we can reproduce a model that's been in production for a year? - ■ *Trap:* "We saved the pickle file." - ■ *Staff:* The registry version links to a run which records git SHA (clean tree), data version (DVC rev / table snapshot), resolved config, container image digest and dependency lock, plus seeds and hardware. DVC remote and image registry have retention policies at least as long as model retention. I validate this quarterly by *actually* re-running a registered model's pipeline in a sandbox and comparing metrics to tolerance.

Q2. MLflow vs W&B — choose for a regulated bank. - ■ *Trap:* "W&B has better charts, so W&B." - ■ *Staff:* Data residency, audit and air-gapped requirements favour self-hosted MLflow (plus its registry) behind SSO, with Postgres and object storage we control. W&B self-managed is possible, but weigh licence, ops cost and integration. I'd give researchers a good UI via MLflow plus dashboards, and standardise on the registry that the CD system reads.

Q3. 5,000 hyperparameter trials swamp the tracking server. What do you do? - ■ *Trap:* "Buy a bigger server." - ■ *Staff:* Reduce write amplification: log metrics every N steps or batch via log_batch; parent/child runs with only summaries on the parent; disable autolog of large artifacts; use early stopping/pruning (Optuna, Hyperband) so fewer trials run to completion; Postgres with indexes and retention policy; consider async logging. Then scale the tracking server horizontally behind a load balancer since it's stateless.

Q4. What's the difference between experiment tracking and the model registry? - ■ *Trap:* "Same thing." - ■ *Staff:* Tracking is an append-only log of *all* attempts for analysis; the registry is a curated, governed list of *candidate-for-deployment* models with versions, aliases, approval state and consumer-facing names. Runs are cheap and numerous; registered versions are few and carry process (reviews, gates, audit).

Q5. Why DVC rather than committing data to Git LFS? - ■ *Trap:* "DVC is open source." - ■ *Staff:* DVC adds pipeline DAG semantics with caching (dvc repro runs only affected stages), parameters and metrics diffing, experiments, and works with any object store without LFS server limits. LFS is only file storage. For very large lakes with branching I'd choose lakeFS or table formats with time travel instead.

Q6. Your validation AUC rose from 0.88 to 0.89 after a code change. Ship it? - ■ *Trap:* "Yes, it improved." - ■ *Staff:* Not enough evidence. Quantify seed variance (e.g., 5 seeds, bootstrap CIs); check per-slice metrics and calibration; ensure the validation set wasn't reused for dozens of tuning decisions; verify no leakage introduced by the change; evaluate on a time-forward holdout; consider the cost (latency/size). Ship if gain exceeds noise, no slice regresses beyond tolerance, and the shadow test agrees.

Q7. How do you handle experiment tracking for notebooks and exploratory work without slowing researchers? - ■ *Trap:* "Ban notebooks." - ■ *Staff:* Provide a thin library with autolog defaults and a notebook template that tags notebooks as stage=exploration. Exploration runs can't be registered; promotion requires converting to the pipeline entrypoint and passing CI. That preserves speed while keeping production lineage clean.

Q8. How do you store and compare very large model artifacts (10+ GB)? - ■ *Trap:* "Log them every epoch to MLflow." - ■ *Staff:* Keep checkpoints in object storage with lifecycle rules (keep best-N + final), log URIs and checksums, use multipart upload, and store only the final/best model as a registry artifact. For comparisons rely on metrics/eval reports, not downloading weights. Consider deltas/LoRA adapters instead of full-model copies.

8. Tips, Tricks & Field Notes

1. Block registration when git.dirty is True; unreproducible champions are the most expensive bug.
2. Log every N steps, not every step; tracking-server bloat is a real outage cause.
3. Compare models across several seeds with confidence intervals, never single points.
4. Select on validation, touch the test set once, and say so out loud in interviews.
5. Log pip freeze and image digest with every run.
6. Use parent/child runs for sweeps; only the parent holds summary metrics.

9. Worked Scenario: The best run cannot be reproduced

Situation. The champion AUC was 0.91 but re-running gives 0.89. Nobody remembers which notebook cell ran.

Steps

1. Check tags: git sha, dirty flag, data hash.
2. Rebuild env from the logged requirements and image digest.

3. Re-run with 5 seeds to measure variance.
4. Decide: noise (document it) or real drift in code/data (fix and re-register).

Outcome. Variance was ~0.01; the team now gates on CI-reported means with intervals.

10. More Interview Questions

Q9. Experiment tracking vs registry? - ■ *Trap:* Same thing. - ■ *Staff:* Tracking is the append-only log of every attempt; the registry is the governed list of deployable candidates with approval state.

Q10. 5,000 trials crush the tracking server. Fix? - ■ *Trap:* Bigger DB. - ■ *Staff:* Batch/log less often, summary-only parents, early-stopping sweeps, Postgres tuning and retention, horizontally scale the stateless server.

Q11. Why DVC over Git LFS? - ■ *Trap:* It is open source. - ■ *Staff:* Pipeline DAG with caching, params/metrics diffs, experiments, and any object-store remote.

11. Where This Module Connects

Direction	Module	What flows
■ Fed by	01 Foundations	reproducibility
■ Fed by	02 Data & Features	versioned data
■ Feeds	04 Registry	runs become versions

12. Interview Playbook: Tips & Tricks

1. Describe reproducibility as a checklist: code SHA, data hash, params, environment, seed, hardware.
2. When given a metric improvement, ask about variance and confidence intervals before celebrating.
3. Mention the tracking-server cost model (rows per run) to show you've operated one.
4. Distinguish exploration runs from candidate runs; it shows governance awareness.
5. State that you select on validation and touch test once.
6. For DVC questions, explain pointers in Git plus content-addressed remote in one sentence.

13. Scenario-Based Evaluation

Each scenario shows what the interviewer is really testing, the answer that loses points, and the answer that earns them.

Scenario 1: Unreproducible champion

Situation. A year-old production model must be rebuilt; the repo has changed and the data bucket was overwritten.

What is being evaluated. Failure analysis and preventive design.

- ■ **Weak answer:** Try to approximate with current data.
- ■ **Strong answer:** 1. Pull the registry version's run tags for git sha, data version and image digest. 2. If data was overwritten, assess whether a snapshot exists; if not, record the gap. 3. Rebuild env from the lockfile and run with fixed seeds. 4. Prevent: immutable content-addressed data, object lock, registration blocked without data hash.

Likely follow-up: *What retention policy do you set for data versus models?*

Scenario 2: Noisy model comparison

Situation. Candidate beats champion by 0.004 AUC on one seed.

What is being evaluated. Statistical judgement.

- ■ **Weak answer:** Ship it.
- ■ **Strong answer:** 1. Run 5-10 seeds; report mean and interval. 2. Bootstrap the holdout; paired comparison. 3. Check slices and calibration. 4. Ship only if the gain exceeds noise and no slice regresses.

Likely follow-up: *How many seeds are enough?*

Scenario 3: Tracking DB meltdown

Situation. A hyperparameter search of 8,000 trials slows the MLflow UI to a crawl.

What is being evaluated. Operational maturity.

- ■ **Weak answer:** Upgrade the instance.
- ■ **Strong answer:** 1. Reduce write volume: log every N steps, use log_batch. 2. Parent run holds summary; children minimal. 3. Early stopping to cut trial count. 4. Index and retention in Postgres; scale stateless server horizontally.

Likely follow-up: *What would you move out of MLflow entirely?*

Scenario 4: Notebook promotion

Situation. A researcher's notebook model scored best. They want it in production tomorrow.

What is being evaluated. Balancing speed and governance.

- ■ **Weak answer:** Block it.
- ■ **Strong answer:** 1. Mark as exploration; it cannot be registered. 2. Pair to convert into the pipeline entrypoint in a day. 3. CI runs tests, gates and data checks. 4. Fast path exists for exploration, not for skipping lineage.

Likely follow-up: *What if they refuse?*

14. Rapid-Fire Round

Question	One-line answer
MLflow backend vs artifact store?	DB for metadata, object store for files.
Why log input example and signature?	Safe serving and schema validation.
DVC pointer file?	.dvc file with hash in Git; data in remote.
W&B vs MLflow?	SaaS UX and sweeps vs self-host and registry.
Seed everything including?	Python, NumPy, framework, dataloader workers.
What breaks determinism on GPU?	Atomic ops and non-deterministic kernels.
dvc repro?	Re-runs only stages with changed deps or params.
What do you tag a run with?	git sha, dirty flag, data hash, image digest.
## 3.15 Statistical Rigor in Model Comparison	

Conceptual Deep Dive

A metric is a random variable. Before claiming that model B beats model A, quantify the noise from four sources: the **random seed** (initialisation, data order, dropout), the **data split**, **hardware non-determinism** (atomics, kernel selection) and the **evaluation sample size**. For a quick estimate, rerun the champion across seeds and splits; the spread is the yardstick any claimed gain must clear. Module 03 section 5 (legacy) covers seed control; this section covers what to do with the residual variance.

Confidence intervals. The nonparametric **bootstrap** resamples evaluation rows with replacement and recomputes the metric; it works for AUC, F1 or any custom metric. For AUC there is also the DeLong approach (named here, not derived), which estimates the variance analytically from the rank structure. For two models scored on the *same* examples, use a **paired bootstrap**: resample row indices once and compute the metric difference on each resample. The pairing cancels example difficulty shared by both models, so the interval on the difference is often several times narrower than two independent intervals. Unpaired tests (a plain t-test across runs evaluated on identical rows) ignore that correlation and are underpowered, while treating test examples as independent when they come from the same user or session overstates precision; resample by group in that case.

Multiple comparisons. Run 50 sweep configurations and the best test score is biased upward by selection (best-of-N). Defend with a validation set for selection and an untouched final test set used once, a corrected threshold (Bonferroni or false discovery rate control) when you must report several comparisons, or at minimum a statement of the selection procedure. A test set reused across hundreds of experiments is adaptively overfit, so rotate or refresh it, and record how many times it has been queried.

Practical significance. Statistical significance is not a launch criterion. Fix the **minimum detectable effect** worth acting on, the cost of switching (retraining, risk, migration), and write the decision rule *before* the experiment: ship if the paired CI lower bound exceeds the minimum effect and latency stays within budget. The seed-variance harness below also answers how many seeds are needed for a target interval width.

Report field	Example content
Metric and effect	mean difference with 95% interval and effect size
Variance	seeds used, std across seeds
Sample	test size, positives, grouping unit
Provenance	hardware, data version, code SHA

Edge cases: a tiny positive class makes AUC intervals wide, so report positive counts; seeds that change data order and initialisation simultaneously blur which source dominates, so vary them separately once; and a test set reused hundreds of times needs a fresh holdout.

Cross-Connection Scenarios

- Online experiment statistics (CUPED, sequential tests) continue in 7.17.
- Statistical assertions inside CI are built in 13.1 and 13.3.
- Frozen evaluation sets and their refresh policy are in 2.17.
- Logging seeds, hardware and data versions per run belongs to 3.16.

Production Anti-Patterns

- Declaring a winner from one seed: a +0.3 point gain is within seed noise and the "improvement" disappears on retrain.
- Selecting the best sweep run and reporting its test score: the number is optimistically biased, and production quality lands below the promised value.
- Running an unpaired t-test across runs evaluated on the same examples: the test has low power, so real gains are rejected and noise is occasionally accepted.

Minimal Working Code

A paired bootstrap returns a confidence interval for the AUC difference between two score vectors (AUC via the rank statistic) and a decision string; the unpaired variant shows why pairing matters.

```
import numpy as np

def auc(s, y):
    r = np.empty(len(s)); r[np.argsort(s, kind="stable")] = np.arange(1, len(s) + 1)
    n1 = int(y.sum()); n0 = len(y) - n1
    return (r[y == 1].sum() - n1 * (n1 + 1) / 2) / (n1 * n0)

def bootstrap_diff(sa, sb, y, n_boot=1000, seed=0, paired=True):
    rng = np.random.default_rng(seed); n = len(y); d = []
    while len(d) < n_boot:
        i = rng.integers(0, n, n)
        j = i if paired else rng.integers(0, n, n)          # unpaired resamples independently
        if 0 < y[i].sum() < n and 0 < y[j].sum() < n:
            d.append(auc(sa[i], y[i]) - auc(sb[j], y[j]))
    lo, hi = (float(q) for q in np.quantile(d, [0.025, 0.975]))
    verdict = "A better" if lo > 0 else "B better" if hi < 0 else "inconcl
usive (CI includes 0)"
    return auc(sa, y) - auc(sb, y), (round(lo, 4), round(hi, 4)), verdict

rng = np.random.default_rng(0)
n = 1500
y = (rng.random(n) < 0.3).astype(int)
base = rng.normal(size=n) + y                               # signal shared by both models
sb = base + rng.normal(0, 0.15, n)
sa = base + 0.1 * y + rng.normal(0, 0.15, n)                # A is slightly better
print("paired  :", bootstrap_diff(sa, sb, y))
```

```
print("unpaired:", bootstrap_diff(sa, sb, y, paired=False))
```

A seed-variance harness trains a tiny logistic regression across seeds (the seed changes initialisation and the train/test split) and reports the mean, std and a 95% interval, plus the seeds needed for a target interval width.

```
import math
import numpy as np

X = np.random.default_rng(123).normal(size=(600, 5))
yall = (X @ np.array([1.0, -1.0, 0.5, 0.0, 0.0]) + np.random.default_rng(124).normal(0, 1.5, 600) > 0)
.astype(float)

def train_eval(seed):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(X)); tr, te = idx[:400], idx[400:]
    w = rng.normal(0, 0.1, 5)                          # seed-dependent init
    for _ in range(150):
        p = 1 / (1 + np.exp(-X[tr] @ w))
        w -= 0.1 * X[tr].T @ (p - yall[tr]) / len(tr)
    return float((((X[te] @ w) > 0) == (yall[te] > 0.5)).mean())

scores = np.array([train_eval(s) for s in range(20)])
mean, std = scores.mean(), scores.std(ddof=1)
half = 1.96 * std / math.sqrt(len(scores))                # normal approx; use a t quantile for small
n
print(f"accuracy {mean:.3f} +/- {half:.3f} (std across seeds {std:.3f})")

def seeds_needed(std, width):
    return math.ceil((2 * 1.96 * std / width) ** 2)

print("seeds for a 0.005-wide interval:", seeds_needed(std, 0.005))
assert seeds_needed(std, 0.005) > len(scores) // 2
```

Interview Angle

Q: A new model has AUC +0.004 over the champion. How do you decide whether it is real? **A:** Compute a paired bootstrap interval of the AUC difference on the same test rows, resampling by user if rows are grouped, and compare with seed-to-seed spread of the champion. Real only if the interval excludes zero and exceeds the minimum effect agreed before the test, after accounting for how many candidates were tried. Trade-off: more seeds and a larger test set cost compute. Measurable check: interval lower bound above the pre-registered margin, and the gain reproduced on a fresh time-forward holdout.

Q: How do you prevent the test set from being overfit by a team running hundreds of sweeps? **A:** Select on validation data only, keep the test set sealed behind a promotion job that logs each query, and cap or rotate it. Report the number of configurations tried and apply a correction when several are compared. Maintain a fresh time-forward holdout for final confirmation. Trade-off: a sealed test slows iteration slightly. Check: the gap between validation and a fresh holdout; a persistent optimistic gap indicates selection bias.

3.16 Metadata, Lineage and Reproducibility at Scale

Conceptual Deep Dive

Reproducibility at scale means a stored, queryable answer to "exactly how was this model made?". The **reproducibility tuple** is: code SHA (plus a dirty-tree flag such as the git.dirty tag used in Module 03 section 2 (legacy)), data version or content hash, resolved config, environment digest (a container image *digest*, not a tag), seed, and hardware. Each lives in a specific place: MLflow params and tags for config and SHA, a DVC lock file or table snapshot id for data, the image digest for the environment, and the model **signature** for the I/O contract. Weights & Biases or MLflow hold the run; neither is the system of record for data.

Metadata stores and lineage. The ML Metadata (MLMD) concept models artifacts, executions and contexts as a graph; OpenLineage standardises run, job and dataset events emitted by orchestrators. The payoff is a graph query: "which models used dataset snapshot X, and which deployments serve those models?" That answer drives incident response (see the lineage graph below) and the consumer registry in 2.15.

Determinism limits. GPU kernels with atomic operations, thread ordering and library versions make bitwise reproduction expensive or impossible. Deterministic modes exist but cost throughput and sometimes disable fast kernels. Define an acceptable **reproduction tolerance** (for example, metric within seed noise, see 3.15) and test it with a nightly rerun, instead of promising bitwise equality.

Data versioning options.

Option	Strength	Cost
DVC (content-addressed files, git pointers)	simple, git-native	large-data ergonomics, remote storage
lakeFS-style branching	cheap branches and commits over object stores	extra service to operate
Table formats with time travel (Delta, Iceberg)	snapshot ids on tables, no copies	retention and vacuum policies must be coordinated
Snapshot tables (copy at train time)	trivial	storage cost, no deduplication

Tracking at scale. Run volume grows faster than anyone plans. Set retention and archival for failed or abandoned runs, sample high-frequency metrics (log every N steps, not every step), size the tracking database for the write rate, and keep the artifacts of registered models out of lifecycle deletion rules. The **audit story** is the acceptance test: pick a prediction served six months ago and reconstruct the request features, the model version, its training data snapshot, code and environment, end to end.

Edge cases: data pulled by query at train time with no snapshot, so "the same query" returns different rows; a reused (mutable) Docker tag; and an artifact deleted by a bucket lifecycle rule while the registry still points at it.

Cross-Connection Scenarios

- Registry aliases and promotion records are in 4.15; consumer notification uses the lineage graph.
- Variance and tolerance definitions come from 3.15.
- Feature versions and backfills (2.15) become nodes in the same graph.
- Audit evidence assembly is in 10.16.

Production Anti-Patterns

- Logging params but not the data version: a "reproducible" run cannot be rebuilt because the table changed, and the champion is unrecoverable.
- Relying on notebook state: hidden cell order and unpinned installs make the result unrepeatable by anyone, including the author.
- Unbounded per-step metric logging: the tracking database slows, the UI times out and the team stops logging, which is worse.

Minimal Working Code

A run manifest records the git SHA, file hashes, an environment freeze hash, seed and params, and a verify function reports exactly which element drifted.

```
import hashlib, json, os, tempfile

def sha256_file(path):
    with open(path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

def build_manifest(git_sha, files, freeze_lines, seed, params):
    return {
        "git_sha": git_sha,
        "files": {p: sha256_file(p) for p in sorted(files)},
        "env_hash": hashlib.sha256("\n".join(sorted(freeze_lines)).encode()).hexdiges
t(),
        "seed": seed,
        "params": params,
    }

def verify(manifest, git_sha, freeze_lines):
```

```
    drift = []
    if git_sha != manifest["git_sha"]:
        drift.append("git_sha")
    drift += [f"file:{p}" for p, h in manifest["files"].items() if not os.path.exists
(p) or sha256_file(p) != h]
    if hashlib.sha256("\n".join(sorted(freeze_lines)).encode()).hexdigest() != manifest["e
nv_hash"]:
        drift.append("environment")
    return drift

with tempfile.TemporaryDirectory() as d:
    data = os.path.join(d, "train.csv")
    with open(data, "w") as f:
        f.write("a,b\n1,2\n")
    freeze = ["numpy==1.26.4", "pandas==2.2.2"]           # in practice: the output
of pip freeze
    m = build_manifest("abc123", [data], freeze, 42, {"lr": 0.1})
    print(verify(m, "abc123", freeze))
    with open(data, "a") as f:
        f.write("3,4\n")                                  # data silently changes
    print(verify(m, "abc123", freeze + ["scipy==1.13.0"]))
    assert verify(m, "abc123", freeze) == [f"file:{data}"]
```

A lineage graph answers which deployed models are affected if a dataset snapshot is corrupt.

```
from collections import defaultdict, deque

class Lineage:
    def __init__(self):
        self.down = defaultdict(set)
        self.kind = {}

    def add(self, node, kind, inputs=()):
        self.kind[node] = kind
        for src in inputs:
            self.down[src].add(node)                      # edge: input -> derived artifact

    def downstream(self, node):
        seen, q = set(), deque([node])
        while q:
            for nxt in self.down[q.popleft()]:
                if nxt not in seen:
                    seen.add(nxt); q.append(nxt)
        return seen

    def affected_deployments(self, dataset):
        return sorted(n for n in self.downstream(dataset) if self.kind[n] == "deployment")

g = Lineage()
g.add("ds:orders@v7", "dataset")
g.add("run:101", "run", ["ds:orders@v7"])
g.add("run:102", "run", ["ds:orders@v7"])
g.add("model:churn@3", "model", ["run:101"])
g.add("model:ltv@1", "model", ["run:102"])
g.add("model:other@9", "model", [])
g.add("deploy:churn-prod", "deployment", ["model:churn@3"])
g.add("deploy:ltv-canary", "deployment", ["model:ltv@1"])
g.add("deploy:other-prod", "deployment", ["model:other@9"])
print(g.affected_deployments("ds:orders@v7"))
assert g.affected_deployments("ds:orders@v7") == ["deploy:churn-prod", "deploy:l
tv-canary"]
```

Interview Angle

Q: Walk through reproducing a model trained a year ago. What fails first in most organisations? **A:** Pull the run's tuple: code SHA, data snapshot id, config, image digest, seed. Usually the data fails first (mutable table, no snapshot, retention expired), then the environment (unpinned dependencies, a deleted base image), then hardware non-determinism. Rebuild, retrain, and accept a metric within the defined tolerance, not bitwise equality. Trade-off: snapshots and long retention cost storage. Measurable check: a quarterly drill that reproduces a random registered model and records the metric difference against the tolerance.

Q: A bad data snapshot was used for three models. How do you find and remediate every affected deployment? **A:** Query the lineage graph downstream from the snapshot to models and then deployments, including indirect paths through features. Prioritise by traffic and risk tier, roll affected deployments back to the last model trained on a good snapshot, retrain on a repaired one and shadow-compare before promotion. Trade-off: speed of rollback versus losing recent improvements. Check: the lineage query returns every deployment, confirmed by diffing serving model ids against the list.

Prev: ← 02 · **Next:** 04 · Model Registry & Governance →